1

```
UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(277) @ 0: reporter [Questa UVM] QUESTA_UVM-1.2.3
UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(278) @ 0: reporter [Questa UVM]  questa_uvm::init(all)
UVM_INFO @ 0: reporter [RNTST] Running test alsu_test...
UVM_INFO ALSU_test.sv(23) @ 40: uvm_test_top [run phase] inside the ALSU test
UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 40: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase

--- UVM Report Summary ---

** Report counts by severity
UVM_INFO :    5
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
** Report counts by id
[Questa UVM]     2
[RNTST]     1
[TEST_DONE]     1
[run phase]     1
** Note: $finish    : C:/questasim64_2021.1/win64/../verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
   Time: 40 ns  Iteration: 54  Instance: /ALSU_top
1
```

2

```
import ALSU_test_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

module ALSU_top();

    bit clk;

    initial begin
        forever #1 clk = ~clk;
    end

    ALSU_if ALSUif(clk);

    ALSU DUT(ALSUif.A,
             ALSUif.B,
             ALSUif.cin,
             ALSUif.serial_in,
             ALSUif.red_op_A,
             ALSUif.red_op_B,
             ALSUif.opcode,
             ALSUif.bypass_A,
             ALSUif.bypass_B,
             ALSUif.clk,
             ALSUif.rst,
             ALSUif.direction,
             ALSUif.leds,
             ALSUif.out);

    initial begin
        uvm_config_db#(virtual ALSU_if)::set(null,"uvm_test_top","ALSU_IF",ALSUif);
        run_test("alsu_test");
    end

endmodule
```

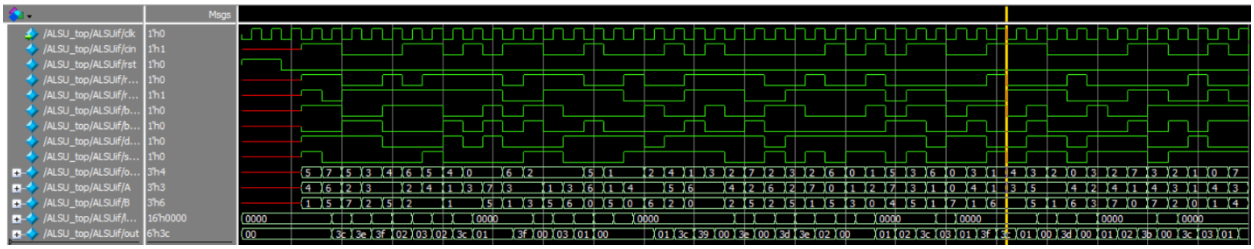
```
package ALSU_test_pkg;
import uvm_pkg::*;
import ALSU_env_pkg::*;
`include "uvm_macros.svh"

    class alsu_test extends uvm_test;
        `uvm_component_utils(alsu_test)

        alsu_env env;
        virtual ALSU_if alsu_test_vif;

        function new(string name = "aslu_test", uvm_component parent = null);
            super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            env = alsu_env::type_id::create("env",this);

            if(!uvm_config_db #(virtual ALSU_if)::get(this,"","ALSU_IF",alsu_test_vif))
                `uvm_fatal("build_phase", "Cannot get interface")

            uvm_config_db #(virtual ALSU_if)::set(this,"*","ALSU_IF",alsu_test_vif);
        endfunction

        task run_phase(uvm_phase phase);
            super.run_phase(phase);
            phase.raise_objection(this);
            #100; `uvm_info("run phase","inside the ALSU test",UVM_MEDIUM);
            phase.drop_objection(this);
        endtask

    endclass


endpackage
```

```
package ALSU_env_pkg;
import uvm_pkg::*;
import ALSU_driver_pkg::*;
`include "uvm_macros.svh"

    class alsu_env extends uvm_env;
        `uvm_component_utils(alsu_env);

        alsu_driver driver;

        function new(string name = "aslu_env", uvm_component parent = null);
            super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            driver = alsu_driver::type_id::create("driver",this);
        endfunction

    endclass

endpackage
```

```
package ALSU_driver_pkg;
import uvm_pkg::*;
`include "uvm_macros.svh"

    class alsu_driver extends uvm_driver;
        `uvm_component_utils(alsu_driver);

        virtual ALSU_if alsu_driver_vif;

        function new(string name = "alsu_driver", uvm_component parent = null);
            super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);

            if(!uvm_config_db#(virtual ALSU_if)::get(this,"","ALSU_IF",alsu_driver_vif))
                `uvm_fatal("build_phase", "Driver cannot get interface")
        endfunction
```

```
        task run_phase(uvm_phase phase);
            // Reset
            alsu_driver_vif.rst = 1;
            repeat (2) @(negedge alsu_driver_vif.clk);
            alsu_driver_vif.rst = 0;

            // Random driving
            forever begin
                @(negedge alsu_driver_vif.clk);

                alsu_driver_vif.A   = $random;
                alsu_driver_vif.B   = $random;
                alsu_driver_vif.cin = $random;
                alsu_driver_vif.serial_in = $random;
                alsu_driver_vif.red_op_A = $random;
                alsu_driver_vif.red_op_B = $random;
                alsu_driver_vif.bypass_A = $random;
                alsu_driver_vif.bypass_B = $random;
                alsu_driver_vif.direction = $random;
                alsu_driver_vif.opcode = $random;
            end
        endtask

    endclass

endpackage
```

```
interface ALSU_if(input logic clk);

    logic cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
    logic [2:0] opcode;
    logic signed [2:0] A, B;
    logic [15:0] leds;
    logic signed [5:0] out;

endinterface //interfacename
```



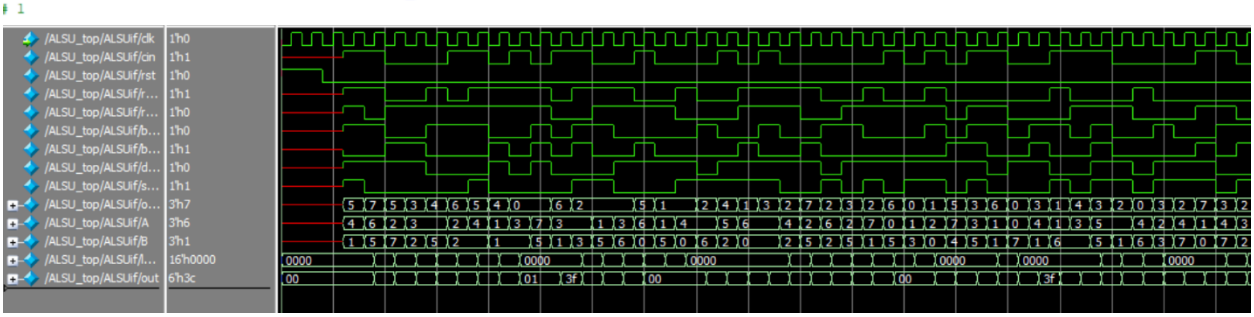

```
# UVM_INFO @ 0: reporter [RNTST] Running test alsu_test...
# UVM_INFO ALSU_test.sv(29) @ 100: uvm_test_top [run phase] inside the ALSU test
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 100: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO :    5
# UVM_WARNING :    0
# UVM_ERROR :    0
# UVM_FATAL :    0
# ** Report counts by id
# [Questa UVM]     2
# [RNTST]     1
# [TEST_DONE]     1
# [run phase]     1
# ** Note: $finish    : C:/questasim64_2021.1/win64/../verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
#    Time: 100 ns  Iteration: 54  Instance: /ALSU_top
# 1
# Break in Task uvm_pkg/uvm_root::run_test at C:/questasim64_2021.1/win64/../verilog_src/uvm-1.1d/src/base/uvm_root.svh line 430
```

3

```
package ALSU_test_pkg;
import uvm_pkg::*;
import ALSU_env_pkg::*;
import ALSU_config_pkg::*;
`include "uvm_macros.svh"

    class alsu_test extends uvm_test;
        `uvm_component_utils(alsu_test)

        alsu_env env;
        ALSU_config ALSU_cfg;

        function new(string name = "aslu_test", uvm_component parent = null);
            super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            env = alsu_env::type_id::create("env",this);
            ALSU_cfg = ALSU_config::type_id::create("ALSU_cfg");

            if(!uvm_config_db #(virtual ALSU_if)::get(this,"","ALSU_IF",ALSU_cfg.ALSU_vif))
                `uvm_fatal("build_phase", "Cannot get interface")

            uvm_config_db #(ALSU_config)::set(this,"*","CFG",ALSU_cfg);
        endfunction

        task run_phase(uvm_phase phase);
            super.run_phase(phase);
            phase.raise_objection(this);
            #100; `uvm_info("run phase","inside the ALSU test",UVM_MEDIUM);
            phase.drop_objection(this);
        endtask

    endclass
```

```
package ALSU_driver_pkg;
import uvm_pkg::*;
import ALSU_config_pkg::*;
`include "uvm_macros.svh"

    class alsu_driver extends uvm_driver;
        `uvm_component_utils(alsu_driver);

        virtual ALSU_if ALSU_vif;
        ALSU_config ALSU_cfg;

        function new(string name = "alsu_driver", uvm_component parent = null);
            super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            ALSU_cfg = ALSU_config::type_id::create("ALSU_cfg");

            if(!uvm_config_db#(ALSU_config)::get(this,"","CFG",ALSU_cfg))
                `uvm_fatal("build_phase", "Driver cannot get interface")
        endfunction

        function void connect_phase(uvm_phase phase);
            super.connect_phase(phase);
            ALSU_vif = ALSU_cfg.ALSU_vif;
        endfunction
```

```
        task run_phase(uvm_phase phase);
            // Reset
            ALSU_vif.rst = 1;
            repeat (2) @(negedge ALSU_vif.clk);
            ALSU_vif.rst = 0;

            // Random driving
            forever begin
                @(negedge ALSU_vif.clk);

                ALSU_vif.A   = $random;
                ALSU_vif.B   = $random;
                ALSU_vif.cin = $random;
                ALSU_vif.serial_in = $random;
                ALSU_vif.red_op_A = $random;
                ALSU_vif.red_op_B = $random;
                ALSU_vif.bypass_A = $random;
                ALSU_vif.bypass_B = $random;
                ALSU_vif.direction = $random;
                ALSU_vif.opcode = $random;
            end
        endtask

    endclass

endpackage
```

```
package ALSU_config_pkg;
import uvm_pkg::*;
`include "uvm_macros.svh"

    class ALSU_config extends uvm_object;
        `uvm_object_utils(ALSU_config)

        virtual ALSU_if ALSU_vif;

    function new(string name = "ALSU_config");
            super.new(name);
    endfunction

    endclass

endpackage
```

```
# UVM_INFO @ 0: reporter [RNTST] Running test alsu_test...
# UVM_INFO ALSU_test.sv(31) @ 100: uvm_test_top [run phase] inside the ALSU test
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 100: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO :    5
# UVM_WARNING :    0
# UVM_ERROR :    0
# UVM_FATAL :    0
# ** Report counts by id
# [Questa UVM]     2
# [RNTST]     1
# [TEST_DONE]     1
# [run phase]     1
# ** Note: $finish    : C:/questasim64_2021.1/win64/../verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
#    Time: 100 ns  Iteration: 54  Instance: /ALSU_top
# 1
```